

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

**Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»**

**Лабораторная работа № 2 по курсу
«Операционные системы и архитектура компьютера»**

Выполнил: К. В. Макаренко

Группа: М8О-204БВ-25

Преподаватель: Н. А. Гапонов

Дата: 03.09.2026

Оценка: _____

Москва, 2026

Условие

Цель работы: Приобретение практических навыков в управлении процессами в ОС и обеспечении обмена данными между процессами посредством каналов.

Задание: Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Вариант: Родительский процесс создает два дочерних процесса. Перенаправление стандартных потоков ввода-вывода показано на картинке выше. Child1 и Child2 можно «соединить» между собой дополнительным каналом. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Процесс child1 и child2 производят работу над строками. Child2 пересылает результат своей работы родительскому процессу. Родительский процесс полученный результат выводит в стандартный поток вывода.

13 вариант) Child1 переводит строки в нижний регистр. Child2 превращает все пробельные символы в символ «_».

Метод решения

Основной код программы находится в main.c — бесконечный цикл, создание трёх pipe, два вызова fork(), обработка ошибок и очистка ресурсов(дескрипторов). Также есть 2 похожих файла с кодом программ для дочерних процессов child1.c и child2.c для последовательной обработки исходной строки.

Описание программы

Структура файлов

Таблица 1 – Файлы проекта

Файл	Назначение
main.c	родительский процесс – создаёт каналы и дочерние процессы
child1.c	дочерний процесс – читает данные из канала и переводит строку в нижний регистр
CmakeLists.txt	сборка трех программ
child2.c	дочерний процесс – читает данные из канала и заменяет пробелы на нижнее подчеркивание

Основные типы данных и функции

Основные функции: error_exit(печатает ошибки в stderr, и завершает выполнение программы со статусом EXIT_FAILURE), read_msg(считывает строку из stdin и возвращает количество считанных байтов), main(создаёт pipe и дочерние процессы),

main в child1.c(переводит строку в нижний регистр), main в child2.c(заменяет пробелы на нижнее подчеркивание).

Основные типы данных: char * — массив символов для хранения строки, int pipe[2] — массив из 2 целых чисел: файловые дескрипторы pipe на чтение и запись, pid_t — псевдоним для целого числа — идентификатор процесса, errno — глобальная переменная, содержащая последнюю ошибку.

Используемые системные вызовы

Таблица 2 – Используемые системные вызовы

Системный вызов	Назначение в программе
int pipe(int fd[2])	создание одностороннего конвейера для передачи строк между процессами
pid_t fork(void)	создание дочернего процесса для преобразования строки
int dup2(int old, int new)	перенаправление стандартного ввода и вывода дочерних процессов на канал
int execl(char *path, const char* arg0, ..., (char *) NULL)	выполнение новой программы, передача списка аргументов, без аргументов PATH

Также: close() - закрытие файлового дескриптора, чтобы ОС знала, что этот номер свободен, kill() - принудительное завершение процесса, waitpid() - приостановка процесса для ожидания завершения дочернего процесса.

Ключевые фрагменты кода

Прочитайте фрагменты, показывающие основные моменты работы программы: создание процессов и потоков, работу с IPC, синхронизацию, обработку ошибок системных вызовов. Перед каждым фрагментом в 1-2 предложениях поясните его роль; в подписи укажите файл. Пример ниже замените своим кодом.

Листинг 1 – Создание канала и дочернего процесса (parent.c)

```
if (msg_len == 0 && feof(stdin)) { // проверка после чтения строки
    break;
}
printf("Прочитано %lu байт\n", msg_len);
int pipe1fd[2], pipe2fd[2], pipe3fd[2]; // подготовка fd
if (pipe(pipe1fd) == -1) error_exit("pipe", strerror(errno)); //ошибка - выход
if (pipe(pipe2fd) == -1) {
    const int saved_errno = errno; // сохранения errno для вывода
    close(pipe1fd[0]); // закрытие ненужных fd
    close(pipe1fd[1]);
    error_exit("pipe", strerror(saved_errno)); //мы дошли до сюда - ошибка
}

...

const pid_t pid1 = fork(); // создание дочернего процесса

if (pid1 == -1) error_exit("fork", strerror(errno)); // перехват ошибки
```

```

...

if (pid1 == 0) { //мы в ребёнке

    dup2(pipe1fd[0], STDIN_FILENO); // подменяем stdin и stdout

    dup2(pipe2fd[1], STDOUT_FILENO);

    close(pipe1fd[0]);

    close(pipe1fd[1]);

    close(pipe2fd[0]);

    close(pipe2fd[1]);

    close(pipe3fd[0]);

    close(pipe3fd[1]);

    execl("./child1", "./child1", nullptr); // выполняем новую программу

    error_exit("execl", strerror(errno)); // если мы тут - ошибка

}

```

Результаты

```

docker run --rm -it --cap-add=SYS_PTRACE -v "$(pwd)":/app os_lab_linux
root@8bc7c648430c:/app# ./main
Hello, OS!
Символ перевода строки найден на индексе 10
Прочитано 11 байт
Результат: hello,_os!
^C
root@8bc7c648430c:/app#

```

Демонстрация работы strace

Программа запущена командой `strace -f ./parent`; полный вывод приведён в приложении Б.

```

read(0, "Hello, OS!\n", 1024) = 11 // чтение строки из stdio
pipe2([3, 4], 0) = 0
pipe2([5, 6], 0) = 0
pipe2([7, 8], 0) = 0 // создание конвейера

```

```

clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0xffff8ee08030) = 148 // создание дочернего процесса

```

```

dup3(3, 0, 0 <unfinished ...> // подмена fd

```

```
close(6 <unfinished ...> // закрытие fd
```

```
write(4, "Hello, OS!\n", 11 <unfinished ...> // запись в pipe строки
```

```
execve("./child1", ["/child1"], 0xffffec7c71d8 /* 8 vars */ <unfinished ...> // работа под капотом exec1
```

```
wait4(166, NULL, 0, NULL)      = 166 // ожидание дочернего процесса
```

Выводы

Реализованы создание дочерних процессов, pipe, обмен данными между ними. Работа с родительским и дочерним процессом в одном и том же коде. Трудность возникла при написании программ для дочерних процессов: было не очевидно, что их надо писать как обычные программы с последующей заменой stdin, stdout, но она была трудностью недолго.

Приложение А. Исходный код программы

Листинг А.1 – Файл main.c

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void error_exit(const char *func_name, const char *error_msg) {
    fprintf(stderr, "[%s]: Ошибка %s\n", func_name, error_msg);
    exit(EXIT_FAILURE);
}

size_t read_msg(char **msg) {
    size_t msg_len = 0;
    const ssize_t read_res = getline(msg, &msg_len, stdin);
    if (read_res == -1) {
        if (feof(stdin))
            printf("Сообщение длиной %lu байт было прочитано со стандартного ввода(stdin): EOF\n",
                msg_len);
        else error_exit("getline", strerror(errno));
        return 0;
    }
    for (ssize_t i = 0; i < read_res; i++) {
        if ((*msg)[i] == '\n') {
            printf("Символ перевода строки найден на индексе %ld\n", i);
        }
    }
    return read_res;
}

int main(void) {
    char *msg = nullptr;
    while (1) {
        const size_t msg_len = read_msg(&msg);
        if (msg_len == 0 && feof(stdin)) {
            break;
        }
        printf("Прочитано %lu байт\n", msg_len);
        int pipe1fd[2], pipe2fd[2], pipe3fd[2];
        if (pipe(pipe1fd) == -1) error_exit("pipe", strerror(errno));
        if (pipe(pipe2fd) == -1) {
            const int saved_errno = errno;
            close(pipe1fd[0]);
            close(pipe1fd[1]);
            error_exit("pipe", strerror(saved_errno));
        }
        if (pipe(pipe3fd) == -1) {
            const int saved_errno = errno;
            close(pipe1fd[0]);
            close(pipe1fd[1]);
            close(pipe2fd[0]);
            close(pipe2fd[1]);
            error_exit("pipe", strerror(saved_errno));
        }
        const pid_t pid1 = fork();
        if (pid1 == -1) error_exit("fork", strerror(errno));
        if (pid1 == 0) {
            dup2(pipe1fd[0], STDIN_FILENO);
            dup2(pipe2fd[1], STDOUT_FILENO);
        }
    }
}
```

```

        close(pipe1fd[0]);
        close(pipe1fd[1]);
        close(pipe2fd[0]);
        close(pipe2fd[1]);
        close(pipe3fd[0]);
        close(pipe3fd[1]);
        execl("./child1", "./child1", nullptr);
        error_exit("execl", strerror(errno));
    }
    const pid_t pid2 = fork();
    if (pid2 == -1) {
        kill(pid1, SIGKILL);
        error_exit("fork", strerror(errno));
    }
    if (pid2 == 0) {
        dup2(pipe2fd[0], STDIN_FILENO);
        dup2(pipe3fd[1], STDOUT_FILENO);
        close(pipe1fd[0]);
        close(pipe1fd[1]);
        close(pipe2fd[0]);
        close(pipe2fd[1]);
        close(pipe3fd[0]);
        close(pipe3fd[1]);
        execl("./child2", "./child2", nullptr);
        error_exit("execl", strerror(errno));
    }
    close(pipe1fd[0]);
    close(pipe2fd[0]);
    close(pipe2fd[1]);
    close(pipe3fd[1]);
    const ssize_t write_res = write(pipe1fd[1], msg, msg_len);
    if (write_res == -1) error_exit("write", strerror(errno));
    close(pipe1fd[1]);
    memset(msg, 0, msg_len);
    const ssize_t read_res = read(pipe3fd[0], msg, msg_len);
    if (read_res == -1) error_exit("read", strerror(errno));
    if (read_res > 0) {
        if (msg[read_res - 1] == '\n') {
            msg[read_res - 1] = '\0';
        } else {
            msg[read_res] = '\0';
        }
    }
    printf("Результат: %s\n", msg);
    close(pipe3fd[0]);
    waitpid(pid1, nullptr, 0);
    waitpid(pid2, nullptr, 0);
}
free(msg);
exit(EXIT_SUCCESS);
}

```

Листинг A.2 – Файл child1.c

```

#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

```

```

#include <sys/types.h>

int main(void) {
    char *msg = nullptr;
    size_t msg_len;

    const ssize_t read_res = getline(&msg, &msg_len, stdin);
    if (read_res == -1) {
        if (feof(stdin)) {
            free(msg);
            return EXIT_SUCCESS;
        }
        perror("[child1]: Ошибка getline");
        free(msg);
        return EXIT_FAILURE;
    }

    for (int i = 0; msg[i] != '\0'; i++) {
        msg[i] = (char) tolower((unsigned char) msg[i]);
    }

    if (printf("%s", msg) < 0) {
        perror("[child1]: Ошибка printf");
        free(msg);
        return EXIT_FAILURE;
    }

    fflush(stdout);

    free(msg);
    return EXIT_SUCCESS;
}

```

Листинг А.3 – Файл child2.c

```

#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <string.h>

int main() {
    char *msg = nullptr;
    size_t msg_len;

    const ssize_t read_res = getline(&msg, &msg_len, stdin);
    if (read_res == -1) {
        if (feof(stdin)) {
            free(msg);
            return EXIT_SUCCESS;
        }
        perror("[child2]: Ошибка getline");
        free(msg);
        return EXIT_FAILURE;
    }

    for (int i = 0; msg[i] != '\0'; i++) {
        if (msg[i] == ' ') msg[i] = '_';
    }
}

```

```
    if (printf("%s", msg) < 0) {  
        perror("[child2 ]: Ошибка printf");  
        free(msg);  
        return EXIT_FAILURE;  
    }  
  
    fflush(stdout);  
  
    free(msg);  
    return EXIT_SUCCESS;  
}
```

Приложение Б. Полный вывод strace

Листинг Б.1 – Вывод strace -f ./parent

```
execve("./main", [ "./main" ], 0xfffffc9f75e58 /* 8 vars */) = 0
brk(NULL) = 0xaaaaaf6026000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0xfffffb9afb000
faccessat(AT_FDCWD, "/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=6587, ...}) = 0
mmap(NULL, 6587, PROT_READ, MAP_PRIVATE, 3, 0) = 0xfffffb9af9000
close(3) = 0
openat(AT_FDCWD, "/usr/lib/aarch64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\3101\2\0\0\0\0"... , 832) =
832
fstat(3, {st_mode=S_IFREG|0755, st_size=1788240, ...}) = 0
mmap(NULL, 1957440, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_DENYWRITE, -1, 0) =
0xfffffb98e7000
mmap(0xfffffb98f0000, 1891904, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0) = 0xfffffb98f0000
munmap(0xfffffb98e7000, 36864) = 0
munmap(0xfffffb9abe000, 28224) = 0
mprotect(0xfffffb9a99000, 81920, PROT_NONE) = 0
mmap(0xfffffb9aad000, 20480, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x1ad000) = 0xfffffb9aad000
mmap(0xfffffb9ab2000, 48704, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -
1, 0) = 0xfffffb9ab2000
close(3) = 0
set_tid_address(0xfffffb9afc388) = 15
set_robust_list(0xfffffb9afc040, 24) = 0
rseq(0xfffffb9afc740, 0x21, 0, 0xd428bc00) = 0
mprotect(0xfffffb9aad000, 12288, PROT_READ) = 0
mprotect(0xaaaaac566f000, 4096, PROT_READ) = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0xfffffb9af7000
mprotect(0xfffffb9b03000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
getrandom("\x81\x50\xf5\x78\xb4\x07\x62\xac", 8, GRND_NONBLOCK) = 8
munmap(0xfffffb9af9000, 6587) = 0
brk(NULL) = 0xaaaaaf6026000
brk(0xaaaaaf6200000) = 0xaaaaaf6200000
fstat(0, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}) = 0
read(0Hello, OS!
, "Hello, OS!\n", 1024) = 11
fstat(1, {st_mode=S_IFREG|0644, st_size=0, ...}) = 0
pipe2([3, 4], 0) = 0
pipe2([5, 6], 0) = 0
pipe2([7, 8], 0) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0xfffffb9afc030) = 16
strace: Process 16 attached
[pid 15] rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
[pid 15] rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
[pid 15] clone(child_stack=NULL,
flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD <unfinished ...>
[pid 16] set_robust_list(0xfffffb9afc040, 24strace: Process 17 attached
) = 0
[pid 15] <... clone resumed>, child_tidptr=0xfffffb9afc030) = 17
[pid 17] set_robust_list(0xfffffb9afc040, 24 <unfinished ...>
```

```

[pid 15] rt_sigprocmask(SIG_SETMASK, [] <unfinished ...>
[pid 16] rt_sigprocmask(SIG_SETMASK, [] <unfinished ...>
[pid 17] <... set_robust_list resumed>) = 0
[pid 16] <... rt_sigprocmask resumed>, NULL, 8) = 0
[pid 15] <... rt_sigprocmask resumed>, NULL, 8) = 0
[pid 15] close(3) = 0
[pid 17] rt_sigprocmask(SIG_SETMASK, [] <unfinished ...>
[pid 15] close(5) = 0
[pid 16] dup3(3, 0, 0 <unfinished ...>
[pid 17] <... rt_sigprocmask resumed>, NULL, 8) = 0
[pid 16] <... dup3 resumed>) = 0
[pid 16] dup3(6, 1, 0) = 1
[pid 15] close(6 <unfinished ...>
[pid 16] close(3 <unfinished ...>
[pid 17] dup3(5, 0, 0 <unfinished ...>
[pid 15] <... close resumed>) = 0
[pid 16] <... close resumed>) = 0
[pid 17] <... dup3 resumed>) = 0
[pid 15] close(8 <unfinished ...>
[pid 16] close(4 <unfinished ...>
[pid 15] <... close resumed>) = 0
[pid 17] dup3(8, 1, 0 <unfinished ...>
[pid 15] write(4, "Hello, OS!\n", 11 <unfinished ...>
[pid 16] <... close resumed>) = 0
[pid 15] <... write resumed>) = 11
[pid 15] close(4) = 0
[pid 17] <... dup3 resumed>) = 1
[pid 15] read(7 <unfinished ...>
[pid 16] close(5 <unfinished ...>
[pid 17] close(3) = 0
[pid 16] <... close resumed>) = 0
[pid 17] close(4) = 0
[pid 16] close(6 <unfinished ...>
[pid 17] close(5) = 0
[pid 17] close(6) = 0
[pid 17] close(7) = 0
[pid 17] close(8 <unfinished ...>
[pid 16] <... close resumed>) = 0
[pid 17] <... close resumed>) = 0
[pid 17] execve("./child2", ["./child2"], 0xfffffe1b81e08 /* 8 vars */
<unfinished ...>
[pid 16] close(7) = 0
[pid 16] close(8) = 0
[pid 16] execve("./child1", ["./child1"], 0xfffffe1b81e08 /* 8 vars */
<unfinished ...>
[pid 17] <... execve resumed>) = 0
[pid 17] brk(NULL) = 0xaaab0cc61000
[pid 17] mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0xfffffbb120000
[pid 17] faccessat(AT_FDCWD, "/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or
directory)
[pid 17] openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
[pid 17] fstat(3, {st_mode=S_IFREG|0644, st_size=6587, ...}) = 0
[pid 17] mmap(NULL, 6587, PROT_READ, MAP_PRIVATE, 3, 0) = 0xfffffbb11e000
[pid 17] close(3) = 0
[pid 17] openat(AT_FDCWD, "/usr/lib/aarch64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC
<unfinished ...>
[pid 16] <... execve resumed>) = 0
[pid 17] <... openat resumed>) = 3

```

```

[pid 17] read(3,
"\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\3101\2\0\0\0\0"... , 832) = 832
[pid 17] fstat(3, {st_mode=S_IFREG|0755, st_size=1788240, ...}) = 0
[pid 17] mmap(NULL, 1957440, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_DENYWRITE, -1,
0) = 0xffffbaf0c000
[pid 16] brk(NULL <unfinished ...>
[pid 17] mmap(0xffffbaf10000, 1891904, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) = 0xffffbaf10000
[pid 17] munmap(0xffffbaf0c000, 16384) = 0
[pid 17] munmap(0xffffbb0de000, 48704) = 0
[pid 16] <... brk resumed> = 0xaaaad0451000
[pid 17] mprotect(0xffffbb0b9000, 81920, PROT_NONE) = 0
[pid 17] mmap(0xffffbb0cd000, 20480, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1ad000) = 0xffffbb0cd000
[pid 17] mmap(0xffffbb0d2000, 48704, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0xffffbb0d2000
[pid 16] mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0xffff82a66000
[pid 16] faccessat(AT_FDCWD, "/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or
directory)
[pid 16] openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
[pid 16] fstat(3, {st_mode=S_IFREG|0644, st_size=6587, ...}) = 0
[pid 16] mmap(NULL, 6587, PROT_READ, MAP_PRIVATE, 3, 0 <unfinished ...>
[pid 17] close(3 <unfinished ...>
[pid 16] <... mmap resumed> = 0xffff82a64000
[pid 17] <... close resumed> = 0
[pid 17] set_tid_address(0xffffbb121388) = 17
[pid 16] close(3 <unfinished ...>
[pid 17] set_robust_list(0xffffbb121040, 24) = 0
[pid 17] rseq(0xffffbb121740, 0x21, 0, 0xd428bc00) = 0
[pid 16] <... close resumed> = 0
[pid 16] openat(AT_FDCWD, "/usr/lib/aarch64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC
<unfinished ...>
[pid 17] mprotect(0xffffbb0cd000, 12288, PROT_READ) = 0
[pid 17] mprotect(0xaaaadc32f000, 4096, PROT_READ) = 0
[pid 17] mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0
<unfinished ...>
[pid 16] <... openat resumed> = 3
[pid 16] read(3,
"\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\3101\2\0\0\0\0"... , 832) = 832
[pid 17] <... mmap resumed> = 0xffffbb11c000
[pid 16] fstat(3, {st_mode=S_IFREG|0755, st_size=1788240, ...}) = 0
[pid 16] mmap(NULL, 1957440, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_DENYWRITE, -1,
0) = 0xffff82852000
[pid 16] mmap(0xffff82860000, 1891904, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0 <unfinished ...>
[pid 17] mprotect(0xffffbb128000, 8192, PROT_READ <unfinished ...>
[pid 16] <... mmap resumed> = 0xffff82860000
[pid 16] munmap(0xffff82852000, 57344) = 0
[pid 16] munmap(0xffff82a2e000, 7744 <unfinished ...>
[pid 17] <... mprotect resumed> = 0
[pid 16] <... munmap resumed> = 0
[pid 17] prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
rlim_max=RLIM64_INFINITY}) = 0
[pid 16] mprotect(0xffff82a09000, 81920, PROT_NONE <unfinished ...>
[pid 17] getrandom("\x8b\x6b\xae\x9b\x16\xc5\xd2\x4b", 8, GRND_NONBLOCK) = 8
[pid 16] <... mprotect resumed> = 0
[pid 17] munmap(0xffffbb11e000, 6587) = 0
[pid 16] mmap(0xffff82a1d000, 20480, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1ad000 <unfinished ...>

```

```

[pid 17] brk(NULL) = 0xaaab0cc61000
[pid 17] brk(0xaaab0ce00000) = 0xaaab0ce00000
[pid 16] <... mmap resumed> = 0xffff82a1d000
[pid 17] fstat(0, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
[pid 16] mmap(0xffff82a22000, 48704, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0 <unfinished ...>
[pid 17] read(0 <unfinished ...>
[pid 16] <... mmap resumed> = 0xffff82a22000
[pid 16] close(3) = 0
[pid 16] set_tid_address(0xffff82a67388) = 16
[pid 16] set_robust_list(0xffff82a67040, 24) = 0
[pid 16] rseq(0xffff82a67740, 0x21, 0, 0xd428bc00) = 0
[pid 16] mprotect(0xffff82a1d000, 12288, PROT_READ) = 0
[pid 16] mprotect(0xaaacfedf000, 4096, PROT_READ) = 0
[pid 16] mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0xffff82a62000
[pid 16] mprotect(0xffff82a6e000, 8192, PROT_READ) = 0
[pid 16] prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
rlim_max=RLIM64_INFINITY}) = 0
[pid 16] getRandom("\x56\xb7\x7c\x3c\x50\x93\x5b\xdb", 8, GRND_NONBLOCK) = 8
[pid 16] munmap(0xffff82a64000, 6587) = 0
[pid 16] brk(NULL) = 0xaaad0451000
[pid 16] brk(0xaaad0600000) = 0xaaad0600000
[pid 16] fstat(0, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
[pid 16] read(0, "Hello, OS!\n", 4096) = 11
[pid 16] fstat(1, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
[pid 16] write(1, "hello, os!\n", 11 <unfinished ...>
[pid 17] <... read resumed>, "hello, os!\n", 4096) = 11
[pid 17] fstat(1 <unfinished ...>
[pid 16] <... write resumed> = 11
[pid 17] <... fstat resumed>, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
[pid 16] exit_group(0 <unfinished ...>
[pid 17] write(1, "hello, os!\n", 11) = 11
[pid 16] <... exit_group resumed> = ?
[pid 15] <... read resumed>, "hello, os!\n", 11) = 11
[pid 17] exit_group(0) = ?
[pid 16] +++ exited with 0 +++
[pid 15] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=16, si_uid=0,
si_status=0, si_utime=0, si_stime=0} ---
[pid 15] close(7) = 0
[pid 17] +++ exited with 0 +++
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=17, si_uid=0, si_status=0,
si_utime=0, si_stime=0} ---
wait4(16, NULL, 0, NULL) = 16
wait4(17, NULL, 0, NULL) = 17

```